Capstone Kick-off Handout — Team DTH

Logistics / Delivery Platform RCA · built on microsoft/OpenRCA · Jira project DEV

> **For the instructor.** This is your run-of-show + the students' hands-on guide for tonight.
> Slides: `docs/slides/kickoff.html` · Repo: https://github.com/ThienHuynhNgoc/OpenRCA_DTH · Board: https://tlearning.atlassian.net/jira/software/projects/DEV/list

---

Part 1 — Run-of-show (about 60 minutes)

Time	What	Material
0–10 min	Welcome + the mission (why RCA, the 11% gap)	Slides 1–3
10–20 min	Our system + the 3-module pipeline	Slides 4–6
20–30 min	How we work (git, PR, conventions)	Slide 7 + Part 5
30–40 min	The roadmap M1–M7	Slide 8 + Part 7
40–55 min	**Hands-on:** clone, run pipeline, open Jira, pick M1 tasks	Slide 9 + Parts 4 & 6
55–60 min	Novelty teaser + Q&A	Slide 10–11 + Part 8

Part 2 — The project in plain words

OpenRCA is a benchmark that asks: *given a software failure and its telemetry (metrics, logs, traces), can a program find the **root cause** — which component broke and why?* The best AI today gets it right only ~11% of the time, so there is real room to contribute.

Our team builds an LLM-assisted RCA system for a **Logistics** scenario using the OpenRCA **Market** dataset. Given a question like:

> On 2021-03-25 between 09:00 and 09:30 the delivery-tracking service showed elevated errors. Find the root cause component and reason.

the system must answer, in OpenRCA's exact format:

```
{ "1": { "root cause occurrence datetime": "2021-03-25 09:21:00",
         "root cause component": "order-service",
         "root cause reason": "high disk I/O read usage" } }
```

Goal of the capstone: a working pipeline → one novel improvement → experiments → a short paper.

Part 3 — Roles & modules (3 devs, 3 modules)

The pipeline has three modules; **each developer owns one**. They communicate only through the shared dataclasses in `src/schemas.py`, so everyone can work in parallel without breaking others.

Module	Suggested owner	What it does	Folder
INPUT	@DuongNg1111	parse the question → time window; load telemetry	`src/input_module/`
PROCESS	@HoangNguyen2803	detect the anomalous component; explain why	`src/process_module/`
OUTPUT	@thanhthanh278	build the OpenRCA JSON; visualize/report	`src/output_module/`

> The leader confirms ownership tonight. Whoever owns a module also owns its Jira tasks.

Part 4 — Environment setup (do this on every laptop)

```
# 1. clone (after accepting the GitHub invite)
git clone git@github.com:ThienHuynhNgoc/OpenRCA_DTH.git
cd OpenRCA_DTH
git switch dev                 # dev is the default working branch

# 2. python env
python3 -m venv .venv
source .venv/bin/activate       # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# 3. prove it works (no dataset needed — runs on mock data)
python -m src.pipeline          # prints a root-cause JSON
python -m pytest -q             # smoke test must pass
```

> Tip: even before `pip install`, `python -m src.pipeline` works (it uses only the standard library).

The real dataset (for M3+) is downloaded later — see `data/DATA.md`.

Part 5 — Git & Pull Request cheat-sheet

`main` and `staging` are **protected — you cannot push to them.** All work flows through PRs.

```
git switch dev && git pull                  # get latest
git switch -c feature/input-load-data       # branch off dev (feature/<module>-<desc>)
# ...edit, then...
git add -p
git commit -m "feat(input): read real metric files"   # Conventional Commits, add 'Refs
git push -u origin feature/input-load-data  # push the FEATURE branch (never main)
```

Then open a **Pull Request into** `dev` on GitHub, fill the template, link the Jira issue, get

1 approval + green CI, and merge. Full rules: `CONTRIBUTING.md`.

Part 6 — Jira: what to do right now (hands-on)

The board is already filled: **7 milestone Epics (M1–M7)** for our team, each with step-by-step

tasks. Tasks are **unassigned** — tonight everyone claims their first one.

Step by step (each student):

1. Log in to **https://tlearning.atlassian.net/jira/software/projects/DEV/list** (Jira project **DEV**).
2. In the board/list, **filter by label** `team-dth` so you only see Team DTH items.
3. Open the Epic `[DTH] M1 — Foundations & Setup`. Read its tasks.
4. **Claim a task:** open it → *Assignee* → assign yourself → drag/move status to **In Progress**.
5. Read the task's **Steps** checklist; do them; tick them off.
6. When done: open your PR, paste its link in the task, move the task to **Done**.

Suggested M1 split for tonight (5 tasks):

- `Accept GitHub invite & clone the repo` — everyone.
- `Install Python env & run the pipeline` — everyone (pair up if stuck).
- `Learn the branch/PR flow with a tiny docs PR` — everyone does their first PR.
- `Download the OpenRCA Market dataset` — one volunteer starts the download.
- `Set up the team board & assign module owners` — **the leader**.

Want to break a task down further? Open the task → **Add child / Create subtask** → write one

small step per subtask (e.g. "install Python", "create venv", "run pytest"). Keep subtasks tiny.

> Rule of thumb: a task should be finishable in a few hours. If it's bigger, make subtasks.

Part 7 — The roadmap (M1 → M7) and what each delivers

Epic	Milestone	Deliverable	Feeds paper section
M1	Foundations & Setup	everyone runs the pipeline; dataset downloaded	—
M2	Literature Review	`RESEARCH.md` filled; reading summaries; novelty shortlist	Related Work
M3	Data & EDA	real loaders + EDA notebook; failure taxonomy	Data
M4	Baseline & Pipeline	real-data pipeline + eval harness + baseline numbers	Method (baseline)
M5	Novelty & Method	the chosen idea implemented + method write-up	Method (novelty)
M6	Experiments & Evaluation	baseline vs proposed vs ablations; figures	Experiments
M7	Paper & Documentation	full draft + reproducibility package + demo	Whole paper

Each Epic in Jira links back to the exact files/folders to touch and to the relevant `docs/` guide.

Part 8 — Where the novelty (the paper) comes from

We will pick **one** of these in M5 and show it beats the baseline in M6:

1. Heuristic + LLM hybrid that triages cheaply and only calls the LLM on hard cases (cut cost & latency).
2. Trace-graph-aware retrieval that walks the service dependency graph to localize the failing service.
3. Domain-transfer framing of microservice RCA as a logistics fulfillment pipeline.

A good capstone paper = a clear baseline + one well-measured improvement + honest error analysis.

Part 9 — FAQ & troubleshooting

- **"I'm not a strong coder."** Good — M1–M3 are mostly setup, reading, and exploring data in a notebook. You'll grow into the code by M4.
- **"`python -m src.pipeline` fails."** Make sure you run it from the repo root and use Python 3.10+.
- **"I can't push to `main`."** That's intended. Push a `feature/*` branch and open a PR into `dev`.
- **"Which task do I take?** Take one from the **M1** Epic with your team's label, assign yourself.
- **"Where is everything?"** Slides: `docs/slides/kickoff.html` · Handbook: `docs/00_capstone_handbook.md`

· This handout: `docs/handouts/kickoff_handout.md` · Board: https://tlearning.atlassian.net/jira/software/projects/DEV/list.

Quick links

- Repo (dev branch): https://github.com/ThienHuynhNgoc/OpenRCA_DTH/tree/dev
- Kickoff slides: https://github.com/ThienHuynhNgoc/OpenRCA_DTH/blob/dev/docs/slides/kickoff.html
- Jira board (filter `team-dth`): https://tlearning.atlassian.net/jira/software/projects/DEV/list
- Capstone handbook: https://github.com/ThienHuynhNgoc/OpenRCA_DTH/blob/dev/docs/00_capstone_handbook.md